Directory Traversal

Kameswari Chebrolu

Department of CSE,
IIT Bombay

Background

- Location of a file described by a filepath
  - E.g. /var/www/images/chebrolu.jpg (in unix)
    - var, www, images are directories starting from root directory “/”
      - This is an absolute path since it starts from root
      - “/” is a path separator also
    - chebrolu.jpg is a file within the images directory

- If one is already within image directory, how can the file be specified?
  - Can be specified as ./chebrolu.jpg
    - This is a relative path
    - Period (.) character denotes the current directory
    - “current” directory depends on the context where the user is currently
- “..” is used to indicate parent directory
  - If one is inside the images folder and do “cd ../../../”, where will one end up?
    - Will end up in root directory

File Access

Type http://example.com/files/shell.php as URL. What happens?

- Web server checks the file's location (likely /var/www/html/files/ folder)
- Apache normally configured to allow access to such files
- Web server will return
  - Content of the file if no mime type associated (or)
  - Output after executing the file if appropriate mime type associated with extension

- Type http://example.com/files/../../../../shell.php in the URL. What happens?
  - What is the location of the file?
    - /shell.php
  - Web server checks the file's folder (/)
  - Apache normally configured to disallow access to files in root folder
  - Web server will return an unauthorized error

```
# Global configuration
ServerRoot "/etc/apache2"

# Listen on port 80
Listen 80

# Server-wide defaults
<Directory />
        Options FollowSymLinks
        AllowOverride None
        Require all denied
</Directory>

<Directory /var/www/>
        Options Indexes FollowSymLinks
        AllowOverride None
        Require all granted
</Directory>

# Logging
ErrorLog ${APACHE_LOG_DIR}/error.log
LogLevel warn
CustomLog ${APACHE_LOG_DIR}/access.log combined

# Include module configurations
IncludeOptional mods-enabled/*.load
IncludeOptional mods-enabled/*.conf

# Include additional directory configurations
IncludeOptional conf-enabled/*.conf

# Virtual hosts
IncludeOptional sites-enabled/*.conf
```

- Type http://example.com/download.php?file=../../../secret.txt in URL. What happens?
- Note we are accessing files via an executable Script called download.php
  - Assume this script locates the file in filesystem and passes the file back to user
- Where is the location of this file?
  - /secret.txt
- Script checks for the requested file
- If script has permission to access root folder, server will return file else gives unauthorized error!
  - Script has whatever user permissions given to web server
  - Many people often install web servers as root → scripts can be tricked to access unauthorized files!

- Why all this relevant?
  - If server-side code runs as root and allows evaluation of relative path, attacker can navigate the filesystem for sensitive files breaking access control!

Directory Traversal Attacks

- Also called Path Traversal
- Allows attackers to access restricted directories outside of the web server's webroot (/var/www/html) directory
  - Depends on how the website access is set up
  - Attacker can impersonate a user associated with "the website" i.e. root or www-data
    - Can access anything the webadmin has access to

- Attack vector: website has some executable that accepts URLs containing parameters describing paths to files
- Attack involves replacing the URL parameter with a relative file path that uses the ../ syntax to “climb out” of webroot

Scenario

https://vulnerable-website.com/loadImage?filename=cat.png

- User supplies the filename
- loadImage script takes the filename parameter and returns the contents of the file
  - May append a base path (e.g. /var/www/html/images/ before fetching the file (/var/www/html/images/cat.png)

Attack: Reveal Sensitive Information

GET https://vulnerable-website.com/loadImage?filename=../../../../etc/passwd HTTP/1.1

- Why ../../../../?
  - ../ sequences moves up the folder /var/www/html/images/ landing in root (/) folder
  - File read is /etc/passwd
  - Can work in windows also (e.g. ..\..\..\..\boot.ini)

Circumventing common defenses

- Defense:
  - Strip or block directory traversal sequences (../../) from the user-supplied filename
  - Check file ends in expected file extension (e.g. jpg or png)

- Circumvention
  - Use ..../
    - ../ is stripped, resulting in ../ which works!
  - URL encoding or double URL encoding
    - ../ same as %2e%2e%2f or %252e%252e%252f
  - Use a null byte to terminate the file path
  - E.g. filename=../../../etc/passwd%00.png
    - %00 is null charcter

Best Practices

- Host static files in CDN or cloud storage if possible
  - They employ best practices like opaque ids
- If writing own code to serve files from disk, use opaque IDs
  - See next slide

- Opaque IDs abstract file paths → prevent direct access to files through predictable filenames
- https://vulnerable-website.com/loadImage?filename=12cGh643Ko
  - 12cGh643Ko is a random ID mapped to the actual file path in database
  - Application looks up 12cGh643Ko in the database
  - Retrieves the corresponding file path and serves (e.g. /var/www/html/images/cat.png)
  - File is served without ever exposing the actual file path

- Attacker can control the filename, but what can be typed to access /etc/passwd?
  - Nothing works since there is no mapping for it in database!

- If using legacy code and cannot refactor, ban path separator characters, including encoded separator characters
  - Validate file names against a regular expression (regex) to filter out anything that looks like path syntax (blacklist)

- Compare against whitelist of permitted values
  - Input contains only alphanumeric characters
  - Canonicalize the path and verify it starts with the expected base directory
    - Canonicalization: shortest absolute path (unique)
    - abc/../abc/file.txt → abc/file.txt
- Use well tested third party libraries for this
- But overall, this is not very secure, attackers often find some way to circumvent

Real Life Examples

Checkout:
https://www.cvedetails.com/vulnerability-list/opdirt-1/directory-traversal.html

Summary

- Path traversals allows attackers to access restricted directories/files
- Defense: Input sanitization not as secure, should follow opaque ids

References

- https://portswigger.net/web-security/file-path-traversal
- https://gist.github.com/SleepyLctl/823c4d29f834a71ba995238e80eb15f9 (Windows cheatsheet)
- https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/Directory%20Traversal/README.md (Payloads)